Untitled

2023-02-25

前言

```
install.packages("tidyverse")
```

安装 tidyverse，tidyverse 是一个包集，里面包含许多个进行数据处理的包如：ggplot2、dplyr、tibble 等。由于这个包集中的包更新较频繁，所以可以使用更新命令 `tidy verse_update()` 对集合中的包进行更新。

```
install.packages(c("nycflights13", "gapminder", "Lahman"))
```

以上三个包包含了 r4ds 这本书中需要用到的数据集。

在学习这本书的过程中或者是将书中所学运用到实际的过程中或碰到很多书中没有解答的问题，这时需要我们自己去从互联网上寻找答案。这时我们可以去 Google 搜索、去 stackoverflow 提问，在向别人求助时，最好的方式是使用 reprex package 创建一个 reprex（a minimal reproducible example）方便与别人交流。

```
reprex::reprex_render()
```

将你的问题代码复制到剪贴板中，然后运行上面的代码就可以在剪贴板中得到一个可以复制到其他地方的 reprex。

第一章 Tibbles with tibble

从上图中可以看到，数据分析有其相对固定的流程，首先是数据的导入（import）、数据的整理（tidy）然后进入到数据 explore 流程，explore 流程包括数据的转换（transform）、数据的可视化（visualize）以及数据建模，最后就是将分析的结果呈现出来进行交流探讨的过程。书本中内容讲述的顺序是首先讲解数据的 explore，然后才是数据大 import、tidy 等过程，但是我在阅读这本书的时候调整了阅读顺序，按照上图中所展示的顺序进行阅读，笔记整理，所以首先学的是 tidyverse 中自定义的数据类型 tibble。

tibble 是一种不同于但是类似于 base R 中 dataframe 的数据结构，这种数据结构是又 tidyverse 中的 tibble 包所提供。

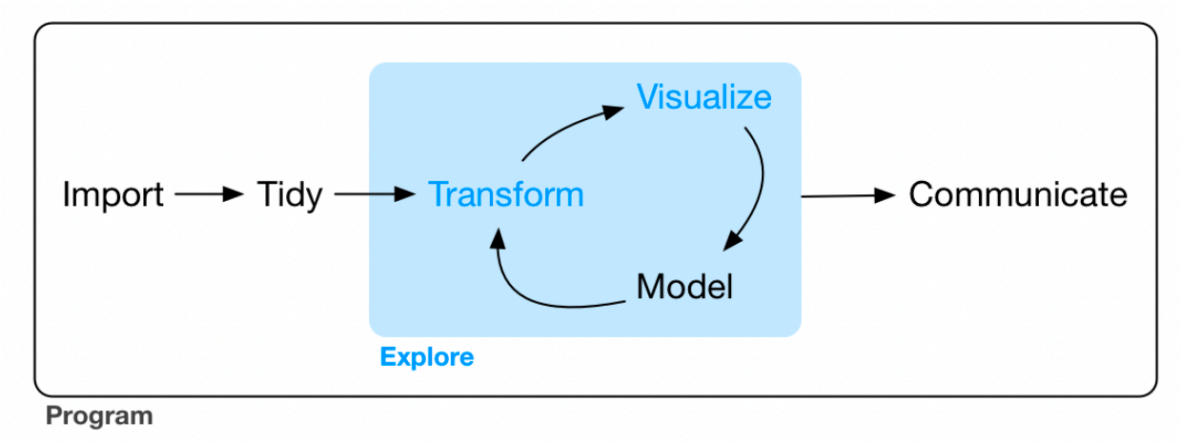


图 1: 数据分析的流程

创建 tibbles

在 tidyverse 中，tibble 是一个内在的属性，所以这个包集中的几乎所有函数的输入输出结果都是 tibble 的形式。但是对于 tidyverse 以外很多包中的函数，都是基于 base r 的 data.frame 的形式，对于这种差别可以使用 `as_tibble()` 函数进行转换。

除此之外，也可以像构建 data.frame 一样手动的建立一个 tibble，第一种方式是使用 `tibble()` 函数。

```
library("tidyverse")
```

```
## -- Attaching packages --------------------------------------- tidyverse 1.3.2 --
## v ggplot2 3.4.0     v purrr   1.0.1
## v tibble  3.1.8     v dplyr   1.1.0
## v tidyr   1.3.0     v stringr 1.5.0
## v readr   2.1.3     v forcats 1.0.0
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
```

```
tibble( x = 1:5, y = 1, z = x^2+y )
```

```
## # A tibble: 5 x 3
##       x     y     z
##   <int> <dbl> <dbl>
## 1     1     1     2
## 2     2     1     5
## 3     3     1    10
## 4     4     1    17
## 5     5     1    26
```

创建 tibble 和创建 data.frame 时存在一些差别：1、tibble 不会去强制的改变数据类型（也就是不会像 data.frame 一样将字符串型的数据转化为因子型。2、tibble 也不会自动改变变量的名名称，也就是说在 tibble 中的变量名称可以不符合 r 的语法规定，如名字以数字或者是特殊字符开头，但是在使用这些不符合语法的名字时，需要将变量名写入 “中。3、tibble 没有行名，只有列名。

手动创建 tibble 的第二种方式是使用 `tribble()` 函数。

```
tribble(
  ~x, ~y, ~z,
  #--/--/---
  "a", 2, 3.6,
  "b", 1, 8.5
)
```

```
## # A tibble: 2 x 3
##   x         y     z
##   <chr> <dbl> <dbl>
## 1 a         2   3.6
## 2 b         1   8.5
```

`tribble()` 创建 tibble 时和使用 `tibble()` 创建 tibble 时存在一些区别，它在创建 tibble 时有其固定的格式，变量的名称使用～符号开头，数据之间以逗号分隔。在书中提到，为了使数据更加的宜都，通常在标题行与数据行之间用 # 符号进行标识。

tibble 和 data.frame 的区别

tibble 和 data.frame 两者之间主要存在两点区别，一个是 printing 的区别，另外一个是 subsetting 的区别。tibble 的输出格式默认只输出前面是个观测值，并且所有的列都会自适应窗口。除了输出列名外，tibble 也会输出每一列数据的数据类型。

有些时候你可能需要输出更多的数据，这时可以通过两种方式来实现，一是修改 `print()` 函数的参数来实现。

```
nycflights13::flights %>%
  print(n = 10, width = Inf)
```

```
## # A tibble: 336,776 x 19
##     year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##    <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>
##  1  2013     1     1      517            515         2      830            819
##  2  2013     1     1      533            529         4      850            830
##  3  2013     1     1      542            540         2      923            850
```

```
## 4  2013     1     1      544            545        -1     1004           1022
## 5  2013     1     1      554            600        -6      812            837
## 6  2013     1     1      554            558        -4      740            728
## 7  2013     1     1      555            600        -5      913            854
## 8  2013     1     1      557            600        -3      709            723
## 9  2013     1     1      557            600        -3      838            846
## 10 2013     1     1      558            600        -2      753            745
##    arr_delay carrier flight tailnum origin dest  air_time distance  hour minute
##        <dbl> <chr>    <int> <chr>   <chr>  <chr>    <dbl>    <dbl> <dbl>  <dbl>
## 1         11 UA        1545 N14228  EWR    IAH        227     1400     5     15
## 2         20 UA        1714 N24211  LGA    IAH        227     1416     5     29
## 3         33 AA        1141 N619AA  JFK    MIA        160     1089     5     40
## 4        -18 B6         725 N804JB  JFK    BQN        183     1576     5     45
## 5        -25 DL         461 N668DN  LGA    ATL        116      762     6      0
## 6         12 UA        1696 N39463  EWR    ORD        150      719     5     58
## 7         19 B6         507 N516JB  EWR    FLL        158     1065     6      0
## 8        -14 EV        5708 N829AS  LGA    IAD         53      229     6      0
## 9         -8 B6          79 N593JB  JFK    MCO        140      944     6      0
## 10         8 AA         301 N3ALAA  LGA    ORD        138      733     6      0
##    time_hour
##    <dttm>
## 1 2013-01-01 05:00:00
## 2 2013-01-01 05:00:00
## 3 2013-01-01 05:00:00
## 4 2013-01-01 05:00:00
## 5 2013-01-01 06:00:00
## 6 2013-01-01 05:00:00
## 7 2013-01-01 06:00:00
## 8 2013-01-01 06:00:00
## 9 2013-01-01 06:00:00
## 10 2013-01-01 06:00:00
## # ... with 336,766 more rows
```

在 `print()` 函数中可以通过修感 n 的值来修改观测值的输出数，可以通过设置 `width=Inf` 来展示所有的列。

另外一种方式是通过全局设置来实现，即设置 `options()` 函数。

```
options(tibble.print_max = n, tibble.print_min = m)
# 调整 tibble() 的输出行数。
options(dplyr.print_min = Inf)
# 总是输出 tibble 的所有行。
```

```
options(tibble.width = Inf)
# 输出 tibble 的所有列数，而不是随着窗口自己进行调整。
```

tibble 和 data.frame 的另外一个区别是提取数据的格式存在差别，提取 tibble 中的某一列数据有很多种方式。

```
df <- tibble( x = runif(5), y = rnorm(5) )
# Extract by name
df$x
```

```
## [1] 0.71477402 0.34932838 0.08112643 0.74881003 0.95779361
```

```
#> [1] 0.434 0.395 0.548 0.762 0.254
df[["x"]]
```

```
## [1] 0.71477402 0.34932838 0.08112643 0.74881003 0.95779361
```

```
#> [1] 0.434 0.395 0.548 0.762 0.254 # Extract by position
df[[1]]
```

```
## [1] 0.71477402 0.34932838 0.08112643 0.74881003 0.95779361
```

```
#> [1] 0.434 0.395 0.548 0.762 0.254
```

第二章 Data Import with readr

在 tidyverse 包集中 readr 包提供来许多函数用来读取数据，当然这个包中的函数只能读取 flat file 特定格式的数据。在这些函数中，最常使用的就是 `read_csv` 函数，这个函数可以用来读取逗号分隔的数据。该函数的第一个参数是数据的存储路径。

```
heights <- read_csv("data/heights.csv")
```

除了提供一个数据的存储路径外，该函数也可以读取一个 inline CSV file。

```
read_csv("a,b,c
         1,2,3
         4,5,6"
  )
```

```
## Rows: 2 Columns: 3
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## dbl (3): a, b, c
##
## i Use `spec()` to retrieve the full column specification for this data.
```

```
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.

## # A tibble: 2 x 3
##       a     b     c
##   <dbl> <dbl> <dbl>
## 1     1     2     3
## 2     4     5     6
```

在默认的情况下，这个函数会将数据集的第一行作为变量名进行读取，要取消这一特性需要向函数中传递参数，可以设置 `skip = n` 来跳过数据中的前面 n 行。

```
read_csv("The first line of metadata
The second line of metadata
        x,y,z
        1,2,3", skip = 2)
```

```
## Rows: 1 Columns: 3
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## dbl (3): x, y, z
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.

## # A tibble: 1 x 3
##       x     y     z
##   <dbl> <dbl> <dbl>
## 1     1     2     3
```

在读取 inline CSV file 时也可以使用 # 符号来将某一行跳过。

```
read_csv("# A comment I want to skip
         x,y,z
         1,2,3", comment = "#")
```

```
## Rows: 1 Columns: 3
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## dbl (3): x, y, z
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.

## # A tibble: 1 x 3
##       x     y     z
```

```
##   <dbl> <dbl> <dbl>
## 1     1     2     3
```

有的时候，数据中没有列名，这时我们可以设置 `col_names = FALSE` 告诉函数不要将第一行作为列名而是自动为每一列生产一个名字从 X1 到 Xn。

```
read_csv("1,2,3\n4,5,6", col_names = FALSE)
```

```
## Rows: 2 Columns: 3
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## dbl (3): X1, X2, X3
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
## # A tibble: 2 x 3
##      X1    X2    X3
##   <dbl> <dbl> <dbl>
## 1     1     2     3
## 2     4     5     6
```

或者是通过向 `col_names` 参数传递一个字符串，给定每一个变量的名字。

```
read_csv("1,2,3\n4,5,6", col_names = c("x", "y", "z"))
```

```
## Rows: 2 Columns: 3
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## dbl (3): x, y, z
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
## # A tibble: 2 x 3
##       x     y     z
##   <dbl> <dbl> <dbl>
## 1     1     2     3
## 2     4     5     6
```

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see http://rmarkdown.

rstudio.com.

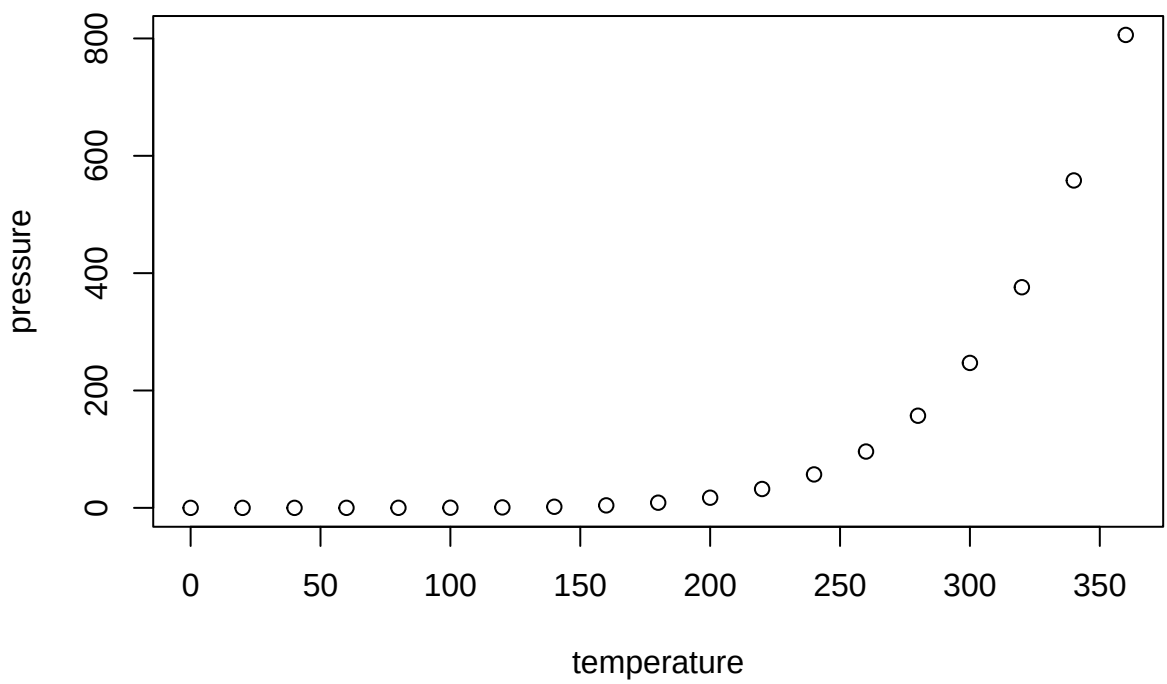
When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

```
summary(cars)
```

```
##      speed           dist
##  Min.   : 4.0   Min.   :  2.00
##  1st Qu.:12.0   1st Qu.: 26.00
##  Median :15.0   Median : 36.00
##  Mean   :15.4   Mean   : 42.98
##  3rd Qu.:19.0   3rd Qu.: 56.00
##  Max.   :25.0   Max.   :120.00
```

Including Plots

You can also embed plots, for example:



Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.